```python
# coding: utf-8
from mindvision.dataset import Mnist
import mindspore.nn as nn
from mindspore import Model
from mindspore.train.callback import ModelCheckpoint, CheckpointConfig
from mindvision.engine.callback import LossMonitor
from mindspore import load_checkpoint, load_param_into_net
import numpy as np
from mindspore import Tensor
import matplotlib.pyplot as plt
from mindspore.common.initializer import Normal

### 定义神经网络
class LeNet5(nn.Cell):
    """
    Lenet网络结构
    """
    def __init__(self, num_class=10, num_channel=1):
        super(LeNet5, self).__init__()
        # 定义所需要的运算
        self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
        self.fc1 = nn.Dense(16 * 5 * 5, 120, weight_init=Normal(0.02))
        self.fc2 = nn.Dense(120, 84, weight_init=Normal(0.02))
        self.fc3 = nn.Dense(84, num_class, weight_init=Normal(0.02))
        self.relu = nn.ReLU()
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()

    def construct(self, x):
        # 使用定义好的运算构建前向网络
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x
network = LeNet5()

# ## 定义损失函数和优化器
# 要训练神经网络模型，需要定义损失函数和优化器函数。
# - 损失函数这里使用交叉熵损失函数`SoftmaxCrossEntropyWithLogits`。
# - 优化器这里使用`Momentum`。

# 定义损失函数
net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')

# 定义优化器函数
net_opt = nn.SGD(network.trainable_params(), learning_rate=0.1, weight_decay=0.0)

# 下载并处理MNIST数据集
download_train = Mnist(path="./mnist", split="train", batch_size=32,
                       repeat_num=1, shuffle=True, resize=32, download=False)

download_eval = Mnist(path="./mnist", split="test", batch_size=32,
                      resize=32, download=False)

dataset_train = download_train.run()
dataset_eval = download_eval.run()


# 训练及保存模型
# 在开始训练之前，MindSpore需要提前声明网络模型在训练过程中是否需要保存中间过程和结果，
# 因此使用`ModelCheckpoint`接口用于保存网络模型和参数，以便进行后续的Fine-tuning（微调）操作。
# 1. 设置模型保存参数
config_ck = CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)

# 2. 应用模型保存参数
ckpoint = ModelCheckpoint(prefix="lenet", directory="./lenet", config=config_ck)

# 3.通过MindSpore提供的`model.train`接口可以方便地进行网络的训练，`LossMonitor`可以监控训练过程中`loss`值的变化
# 初始化模型参数
model = Model(network, loss_fn=net_loss, optimizer=net_opt, metrics={'accuracy'})

# 4.训练网络模型
model.train(50, dataset_train, callbacks=[ckpoint, LossMonitor(0.01, 1875)])
```

```python
# 训练过程中会打印loss值，loss值会波动，但总体来说loss值会逐步减小，精度逐步提高。每个人运行的loss值有一定随机性，不一定完全相同。

# 5.通过模型运行测试数据集得到的结果，验证模型的泛化能力：
# 5.1 使用`model.eval`接口读入测试数据集。
acc = model.eval(dataset_eval)
print("{}".format(acc))



# 可以在打印信息中看出模型精度数据，示例中精度数据达到95%以上，模型质量良好。随着网络迭代次数增加，模型精度会进一步提高。


# 5.2 使用保存后的模型参数进行推理。
# ## 加载模型
# 加载已经保存的用于测试的模型
param_dict = load_checkpoint("./lenet/lenet-1_1875.ckpt")
# 加载参数到网络中
load_param_into_net(network, param_dict)

#加载测试集
mnist = Mnist("./mnist", split="test", batch_size=6, resize=32)
dataset_infer = mnist.run()
ds_test = dataset_infer.create_dict_iterator()
data = next(ds_test)
images = data["image"].asnumpy()
labels = data["label"].asnumpy()

# 可视化该测试集
plt.figure()
for i in range(1, 7):
    plt.subplot(2, 3, i)
    plt.imshow(images[i-1][0], interpolation="None", cmap="gray")
plt.show()

# 使用函数model.predict预测image对应分类
output = model.predict(Tensor(data['image']))
predicted = np.argmax(output.asnumpy(), axis=1)

# 输出预测分类与实际分类
print(f'Predicted: "{predicted}", Actual: "{labels}"')

#从上面的打印结果可以看出，预测值与目标值完全一致。
```